Adatbázisok 2 – ZH Túlélőkészlet & Jegyzet

- creating a schema
  - táblaterek
  - userses
  - táblákok
  - adatok beszurkálása
  - View-ok
- A Két Legutóbbi Éles ZH Komplett Megoldókulcsa (Puskák gyöngye)
  - 1. ZH Megoldókulcs: TERMEK és KATEGORIA (zh1.pdf)
  - 2. ZH Pót Megoldókulcs: KONYVTAR, KONYV és KOLCSONZES (zh1_pot.pdf)
  - Táblamódosítások (ALTER TABLE) - Ha elcseszted a táblát és nem akarod elölről kezdeni
  - DML alapok: UPDATE és DELETE (Mert hibázni emberi dolog)
  - Lekérdezések Mesterfokon (SELECT Nagyágyú)
  - Neat Tricks & Életmentő Oracle Függvények (ZH Túlélőcsomag)
  - Szekvenciák (Sequences) - Automatikus azonosító számlálók
  - Indexek (Ha száguldani kell a WHERE-ben)
  - Tranzakciókezelés: COMMIT, ROLLBACK és Mentési Pontok
  - Adatszótár Puskák (Data Dictionary Cheat Sheet)
  - A Leggyakoribb Oracle Hibák és Megoldásuk (Pánikgomb)
- PL/SQL Túlélőcsomag (Órai Anyag & 2. ZH Előzetes)
  - 1. Az Aranyszabály: Kapcsold be a kimenetet!
  - 2. Névtelen blokk (Anonymous Block) szerkezete
  - 3. A Szentséges SELECT ... INTO és a Két Mumus
  - 4. Tárolt Eljárások (PROCEDURE) - IN és OUT paraméterekkel
  - 5. Tárolt Függvények (FUNCTION)
  - 6. PL/SQL Csomagok (PACKAGE) - A kód modularitása
  - 7. Audit és Történeti Naplózás Eljárásban (A `change_owner` Minta)
- ZH Környezet & Auditáló Túlélési Szabályzat (Hogyan ne ess ki a játékból az első 5 percben)

creating a schema

Igen furcsa mód az oracle-nel a séma és az userek ugyanazok(?) vagy legalábbis annak tűnnek nekem...

táblaterek

```
CREATE TABLESPACE táblatér_név
DATAFILE 'táblatér_név.dbf'
SIZE 10M AUTOEXTEND on;
```

```
CREATE TEMPORARY TABLESPACE ideiglenes_táblatér_név
tempfile 'ideiglenes_táblatér_név.dat'
SIZE 10M AUTOEXTEND on;
```

userses

```
CREATE USER felhasználónév IDENTIFIED BY jelszo
DEFAULT TABLESPACE táblatér_név
TEMPORARY TABLESPACE ideiglenes_táblatér_név;

GRANT CREATE SESSION TO felhasználónév;
GRANT CONNECT, RESOURCE TO felhasználónév;
GRANT CREATE TABLE TO felhasználónév;
GRANT CREATE VIEW TO felhasználónév;
GRANT UNLIMITED TABLESPACE TO felhasználónév;
```

> **Tipp a Pót ZH-hoz és az órai anyaghoz: Létező séma biztonságos eldobása és újrahúzása (A beugró megmentője):** Ha a feladat úgy szól: *"Hozz létre egy KONYVTAR sémát..., ha már létezik a séma, akkor előtte dobd el"*, a sima `CREATE USER` azonnal elhasal `ORA-01920: user name conflicts with another user name` hibával. Rendszergazdaként (SYS / SYSTEM) futtass le egy dinamikus PL/SQL törlőblokkot, ami csak akkor dobja el, ha tényleg létezik:

```
-- 0. Feladat: Séma eldobása ha már létezik, majd újrahúzása jogokkal (class_work/00_sys.s
        ql minta)
DECLARE
  v_count NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_count FROM all_users WHERE username = UPPER('felhasználónév');
  IF v_count = 1 THEN
    EXECUTE IMMEDIATE 'DROP USER felhasználónév CASCADE';
  END IF;
END;
/

CREATE USER felhasználónév IDENTIFIED BY jelszo
  DEFAULT TABLESPACE users
  TEMPORARY TABLESPACE temp
  QUOTA UNLIMITED ON users;

GRANT CREATE SESSION TO felhasználónév;
GRANT CREATE TABLE TO felhasználónév;
GRANT CREATE VIEW TO felhasználónév;
GRANT CREATE SEQUENCE TO felhasználónév;
GRANT CREATE PROCEDURE TO felhasználónév;

-- Ha a saját felhasználód alatt akarsz ténykedni anélkül, hogy mindig kiírnád a nevét:
ALTER SESSION SET CURRENT_SCHEMA = felhasználónév;
```

> ⚠️ **Fontos PDB trükk:** Ha a fenti `CREATE USER`-nél `ORA-65096: invalid common user name` hibát kapsz, mert a gyökér (CDB) konténerben vagy, először válts át a PDB-re: `ALTER SESSION SET CONTAINER = XEPDB1;`

táblákok

```
CREATE TABLE tábla_név (
    azonosító NUMBER,
    megszólítás VARCHAR2(100) CONSTRAINT nn_megszólítás NOT NULL,
    dátum DATE DEFAULT SYSDATE,
    CONSTRAINT pk_tábla_név PRIMARY KEY (azonosító),
    CONSTRAINT uq_megszólítás UNIQUE (megszólítás)
);
```

> **Tipp kényszerekhez:** - Oracle-ben a `NOT NULL`-t oszlopszinten szokás megadni (vagy táblaszinten: `CHECK (oszlop IS NOT NULL)`). - A kényszerneveknek az egész sémában egyedinek kell lenniük (nem lehet mindkét táblában ugyanaz a kényszernév)! - **Szöveghossz kényszer (Pót ZH kedvenc):** Ha megkövetelik, hogy pl. az olvasó neve legalább 10 karakter hosszú legyen: `CONSTRAINT ck_olvaso_nev CHECK (LENGTH(olvaso_neve) >= 10)` - **Tábla és oszlop megjegyzések (Kommentek az órai anyagból):** Hogy te is és a vizsgáztató is értse mi micsoda: ```sql COMMENT ON TABLE tábla_név IS 'Gyerekek és csecsemők adatbázisa'; COMMENT ON COLUMN tábla_név.megszólítás IS 'A lurkó anyakönyvezett becsületes neve';
>
> – Ellenőrzés adatszótárból: SELECT table_name, comments FROM user_tab_comments WHERE table_name = 'TÁBLA_NÉV'; SELECT column_name, comments FROM user_col_comments WHERE table_name = 'TÁBLA_NÉV'; ```

```sql
CREATE TABLE másik_tábla_név (
    azonosító NUMBER,
    megszólítás VARCHAR2(100) CONSTRAINT nn_masik_megszólítás NOT NULL,
    nem_nulla_szám NUMBER,
    kŐlső_kulcs_id NUMBER,
    CONSTRAINT pk_másik_tábla_név PRIMARY KEY (azonosító),
    CONSTRAINT uq_masik_megszólítás UNIQUE (megszólítás),
    CONSTRAINT ck_nem_nulla_szám CHECK (nem_nulla_szám > 0),
    CONSTRAINT fk_kŐlső_kulcs_id FOREIGN KEY (kŐlső_kulcs_id) REFERENCES tábla_név(azonosí
        tó)
);
```

adatok beszurkálása

```sql
INSERT INTO tábla_név VALUES (
    1, 'Lökött Pista', TO_DATE('1994/12/16 12:00:00', 'yyyy/mm/dd hh:mi:ss')
);
```

```sql
-- Vesszővel elválasztott multi-row INSERT (Oracle 23ai-tól támogatott,
-- régebbi Oracle 11g-21c verziókban használj inkább INSERT ALL-t lásd lejjebb!):
INSERT INTO másik_tábla_név (azonosító, megszólítás, kŐlső_kulcs_id)
VALUES
    (67, 'gumi kaki játék', 1),
    (69, 'építő kockák', 1);
```

```sql
INSERT ALL
    INTO tábla_név VALUES (2, 'Lökött Gábor', TO_DATE('1997/12/16 12:00:00', 'yyyy/mm/dd h
        h:mi:ss'))
    INTO tábla_név VALUES (3, 'Néni Babi', TO_DATE('2026/09/30 12:00:00', 'yyyy/mm/dd hh:m
        i:ss'))
SELECT * FROM dual;
COMMIT;
```

> ☠️ **DÁTUMCSAPDA a ZH-n: "Aktuális dátum óra, perc és másodperc nélkül!"** Ha simán beírod, hogy `SYSDATE`, abban másodpercre pontosan benne lesz a mostani idő (pl. `2026-10-19 21:15:32`). A ZH feladatok kifejezetten kérik: *óra, perc és másodperc nélkül*! Ilyenkor **KÖTELEZŐ a `TRUNC`**: - Kivétel dátuma (ma 00:00:00): `TRUNC(SYSDATE)` - Lejárat dátuma (jövőbeli dátum idő nélkül): `TRUNC(SYSDATE) + 14` vagy `ADD_MONTHS(TRUNC(SYSDATE), 1)`

```
-- Két tábla egyidejű feltöltése egyetlen utasítással (Pót ZH 3. feladat mintája):
INSERT ALL
    INTO konyv (id, cim, kategoria, ar) VALUES (1, 'Dűne', 'Sci-fi', 4500)
    INTO konyv (id, cim, kategoria, ar) VALUES (2, 'Alapítvány', 'Sci-fi', 3800)
    INTO konyv (id, cim, kategoria, ar) VALUES (3, 'A Gyűrűk Ura', 'Fantasy', 6500)
    INTO kolcsonzes (id, konyv_id, olvaso_neve, kivetel_datuma, lejarat_datuma)
        VALUES (101, 1, 'Kovács Jánoska', TRUNC(SYSDATE), TRUNC(SYSDATE) + 14)
    INTO kolcsonzes (id, konyv_id, olvaso_neve, kivetel_datuma, lejarat_datuma)
        VALUES (102, 2, 'Tóth László Béla', TRUNC(SYSDATE), ADD_MONTHS(TRUNC(SYSDATE), 1))
    INTO kolcsonzes (id, konyv_id, olvaso_neve, kivetel_datuma, lejarat_datuma)
        VALUES (103, 1, 'Szabó-Eszterházy Nóra', TRUNC(SYSDATE) - 10, TRUNC(SYSDATE) + 4)
SELECT * FROM dual;
COMMIT;
```

Másolat tábla és idempotens MERGE beszúrás

1. Üres másolat tábla létrehozása manuális oszlopfelsorolás nélkül (talán)

```
CREATE TABLE tábla_név_másolat AS
SELECT *
FROM tábla_név
WHERE 1 = 0;
```

2. Idempotens adatfeltöltés MERGE utasítással: (remélem 😅)

```
MERGE INTO tábla_név_másolat target
USING tábla_név source
ON (target.azonosító = source.azonosító)
WHEN NOT MATCHED THEN
INSERT (azonosító, megszólítás, dátum)
VALUES (source.azonosító, source.megszólítás, source.dátum);
COMMIT;
```

Válogatás új táblákba mező érték alapján (itten pölö dátum szerint)

1. Először isletrehozunk 2 táblát amibe nincs semmi, de rendelkeznek mindennel, ami kell, hogy az eredeti táblából áttuggyuk bele baszni a megfelelően válogatott sorokat!

```
CREATE TABLE regi_tábla_név AS
SELECT * FROM tábla_név WHERE 1 = 0;

CREATE TABLE új_tábla_név AS
SELECT * FROM tábla_név WHERE 1 = 0;
```

2. Most kéne válogatni valahogy...

```
INSERT FIRST
    WHEN dátum < ADD_MONTHS(SYSDATE, -6) THEN
        INTO regi_tábla_név (azonosító, megszólítás, dátum)
        VALUES (azonosító, megszólítás, dátum)
    ELSE
        INTO új_tábla_név (azonosító, megszólítás, dátum)
        VALUES (azonosító, megszólítás, dátum)
SELECT azonosító, megszólítás, dátum
FROM tábla_név;

COMMIT;
```

3. Ellenőrizni is érdemes:

```
SELECT 'Régi kölkök' AS azonosító, megszólítás, dátum FROM regi_tábla_név
UNION ALL
SELECT 'Új kölkök' AS azonosító, megszólítás, dátum FROM új_tábla_név;
```

Kicsitt késő tisztázni, de a tábla_név-ben gyerekek szerepelnek a példák szerint, a másik_tábla_név-ben pedig meg hozzájuk rendelt játékok vannak (gyerek:játék = 1:N)

View-ok

Csoportosító view

Hogy hívják azt az állóvizet amelyik legjobban összehozza az embereket: Csoportosí-tó!



ROLF COPTER

```
CREATE OR REPLACE VIEW v_tábla_név_másik_tábla_név_osszesito AS
SELECT
    k.azonosító                                                    AS tábla_név_id,
    k.megszólítás                                                AS tábla_név_nev,
    COUNT(t.id)                                                 AS másik_tábla_név_szama,
    LISTAGG(t.megszólítás, '|') WITHIN GROUP (ORDER BY t.megszólítás)   AS másik_tábla_név
        _nevei
FROM kategoria k
INNER JOIN termek t ON k.azonosító = t.tábla_név_id
GROUP BY k.azonosító, k.megszólítás;
```

****Ellenőrizés:****

```
SELECT * FROM v_tábla_név_másik_tábla_név_osszesito;
```

A Két Legutóbbi Éles ZH Komplett Megoldókulcsa (Puskák gyöngye)

Itt van feketén-fehéren a két lefutott éles zárthelyi (zh1.pdf és zh1_pot.pdf) összes feladatának 100%-osan tesztelt, a beugrótól az utolsó pontig működő megoldása.

---

1. ZH Megoldókulcs: TERMEK és KATEGORIA (zh1.pdf)

1. Feladat: TERMEK séma létrehozása (Beugró)

```
-- Rendszergazdaként (SYS / SYSTEM):
CREATE USER termek IDENTIFIED BY simplePassword
  DEFAULT TABLESPACE users
  QUOTA UNLIMITED ON users;

GRANT CREATE SESSION TO termek;
GRANT CREATE TABLE TO termek;
GRANT CREATE VIEW TO termek;
GRANT CREATE SEQUENCE TO termek;

ALTER SESSION SET CURRENT_SCHEMA = termek;
```

2-3. Feladat: Táblák és kényszerek létrehozása (2 pont)

```
-- 1. Szülő tábla (KATEGORIA):
CREATE TABLE kategoria (
    id NUMBER,
    megnevezes VARCHAR2(100) CONSTRAINT nn_kat_megnevezes NOT NULL,
    leiras VARCHAR2(500),
    CONSTRAINT pk_kategoria PRIMARY KEY (id)
);

-- 2. Gyerek tábla (TERMEK):
CREATE TABLE termek (
    id NUMBER,
    nev VARCHAR2(100) CONSTRAINT nn_termek_nev NOT NULL,
    ar NUMBER,
    reg_datum DATE DEFAULT SYSDATE,
    kategoria_id NUMBER,
    CONSTRAINT pk_termek PRIMARY KEY (id),
    CONSTRAINT uq_termek_nev UNIQUE (nev),
    CONSTRAINT ck_termek_ar CHECK (ar > 0),
    CONSTRAINT fk_termek_kategoria FOREIGN KEY (kategoria_id) REFERENCES kategoria(id)
);
```

4. Feladat: 5-5 rekord feltöltése egyszerre (1 pont)

```
INSERT ALL
    -- 5 Kategória
    INTO kategoria (id, megnevezes, leiras) VALUES (1, 'Autóalkatrész', 'Jármű alkatrészek
        és tartozékok')
    INTO kategoria (id, megnevezes, leiras) VALUES (2, 'Elektronika', 'Okostelefonok, lapt
        opok, kütyük')
    INTO kategoria (id, megnevezes, leiras) VALUES (3, 'Könyvek', 'Szépirodalom és műszaki
        szakkönyvek')
    INTO kategoria (id, megnevezes, leiras) VALUES (4, 'Sportszerek', 'Kültéri és beltéri
        sportfelszerelések')
    INTO kategoria (id, megnevezes, leiras) VALUES (5, 'Antik bútor', 'Klasszikus és felúj
        ított bútorok')
    -- 5 Termék (szinkronban a kategóriákkal)
    INTO termek (id, nev, ar, reg_datum, kategoria_id)
        VALUES (101, 'Fékbetét garnitúra', 18500, TO_DATE('2023-01-15', 'YYYY-MM-DD'), 1)
    INTO termek (id, nev, ar, reg_datum, kategoria_id)
        VALUES (102, 'Vezeték nélküli fülhallgató', 24900, TO_DATE('2024-08-10', 'YYYY-MM-
        DD'), 2)
    INTO termek (id, nev, ar, reg_datum, kategoria_id)
        VALUES (103, 'Oracle SQL mesterkurzus', 9500, TO_DATE('2023-05-20', 'YYYY-MM-DD'),
        3)
    INTO termek (id, nev, ar, reg_datum, kategoria_id)
        VALUES (104, 'Focilabda FIFA Pro', 12000, TO_DATE('2024-09-01', 'YYYY-MM-DD'), 4)
    INTO termek (id, nev, ar, reg_datum, kategoria_id)
        VALUES (105, 'Olajszűrő prémium', 4500, TO_DATE('2024-07-25', 'YYYY-MM-DD'), 1)
SELECT * FROM dual;

COMMIT;
```

5. Feladat: Üres másolat (CTAS) + Idempotens beszúrás MERGE-dzsel (1.5 pont)

```
-- Üres másolat létrehozása manuális oszlopfelsorolás nélkül:
CREATE TABLE kategoria_masolat AS
SELECT * FROM kategoria WHERE 1 = 0;

-- Idempotens feltöltés (akárhányszor lefuttatható hiba nélkül, nincs DELETE/TRUNCATE):
MERGE INTO kategoria_masolat target
USING kategoria source
ON (target.id = source.id)
WHEN NOT MATCHED THEN
    INSERT (id, megnevezes, leiras)
    VALUES (source.id, source.megnevezes, source.leiras);

COMMIT;
```

6. Feladat: Szétválogatás félévnél régebbi / újabb alapján (1.5 pont)

```
-- Két üres tábla létrehozása a termék szerkezetével:
CREATE TABLE regi_termek AS SELECT * FROM termek WHERE 1 = 0;
CREATE TABLE uj_termek   AS SELECT * FROM termek WHERE 1 = 0;

-- Szétválogató beszúrás félév (ADD_MONTHS(SYSDATE, -6)) alapján:
INSERT FIRST
    WHEN reg_datum < ADD_MONTHS(SYSDATE, -6) THEN
        INTO regi_termek (id, nev, ar, reg_datum, kategoria_id)
        VALUES (id, nev, ar, reg_datum, kategoria_id)
    ELSE
        INTO uj_termek (id, nev, ar, reg_datum, kategoria_id)
        VALUES (id, nev, ar, reg_datum, kategoria_id)
SELECT id, nev, ar, reg_datum, kategoria_id
FROM termek;

COMMIT;
```

7. Feladat: Összesítő nézet LISTAGG-gal és darabszámmal (2 pont)

```
CREATE OR REPLACE VIEW v_kategoria_termek_osszesito AS
SELECT
    k.id                                                    AS kategoria_id,
    k.megnevezes                                            AS kategoria_nev,
    COUNT(t.id)                                             AS termekek_szama,
    LISTAGG(t.nev, '|') WITHIN GROUP (ORDER BY t.nev)       AS termekek_nevei
FROM kategoria k
INNER JOIN termek t ON k.id = t.kategoria_id
GROUP BY k.id, k.megnevezes;

-- Ellenőrzés:
SELECT * FROM v_kategoria_termek_osszesito;
```

8. Feladat: Utolsó 'A' betűs kategória és termékei LEFT JOIN-nal (2 pont)

```
WITH kivalasztott_kategoria AS (
    SELECT *
    FROM (
        SELECT id, megnevezes, leiras
        FROM kategoria
        WHERE UPPER(megnevezes) LIKE '%A%'
        ORDER BY megnevezes DESC
    )
    WHERE ROWNUM = 1
)
SELECT
    k.id            AS kategoria_id,
    k.megnevezes    AS kategoria_nev,
    k.leiras        AS kategoria_leiras,
    t.id            AS termek_id,
    t.nev           AS termek_nev,
    t.ar            AS termek_ar,
    t.reg_datum     AS termek_reg_datum
FROM kivalasztott_kategoria k
LEFT JOIN termek t ON k.id = t.kategoria_id;
```

---

2. ZH Pót Megoldókulcs: KONYVTAR, KONYV és KOLCSONZES (zh1_pot.pdf)

0. Feladat: KONYVTAR séma eldobása ha van, majd újraalkotása (Beugró)

```
-- Rendszergazdaként (SYS / SYSTEM):
DECLARE
  v_count NUMBER;
BEGIN
  SELECT COUNT(*) INTO v_count FROM all_users WHERE username = 'KONYVTAR';
  IF v_count = 1 THEN
    EXECUTE IMMEDIATE 'DROP USER konyvtar CASCADE';
  END IF;
END;
/

CREATE USER konyvtar IDENTIFIED BY konyvtarJelszo
  DEFAULT TABLESPACE users
  QUOTA UNLIMITED ON users;

GRANT CREATE SESSION TO konyvtar;
GRANT CREATE TABLE TO konyvtar;
GRANT CREATE VIEW TO konyvtar;
GRANT CREATE SEQUENCE TO konyvtar;

ALTER SESSION SET CURRENT_SCHEMA = konyvtar;
```

1-2. Feladat: KONYV és KOLCSONZES táblák + megszorítások (2 pont)

```
-- 1. KONYV tábla:
CREATE TABLE konyv (
    id NUMBER,
    cim VARCHAR2(200) CONSTRAINT nn_konyv_cim NOT NULL,
    kategoria VARCHAR2(100),
    ar NUMBER,
    CONSTRAINT pk_konyv PRIMARY KEY (id),
    CONSTRAINT uq_konyv_cim UNIQUE (cim)
);

-- 2. KOLCSONZES tábla:
CREATE TABLE kolcsonzes (
    id NUMBER,
    konyv_id NUMBER CONSTRAINT nn_kolcsonzes_konyv_id NOT NULL,
    olvaso_neve VARCHAR2(150) CONSTRAINT nn_kolcsonzes_olvaso_nev NOT NULL,
    kivetel_datuma DATE,
    lejarat_datuma DATE,
    CONSTRAINT pk_kolcsonzes PRIMARY KEY (id),
    CONSTRAINT fk_kolcsonzes_konyv FOREIGN KEY (konyv_id) REFERENCES konyv(id),
    CONSTRAINT ck_kolcsonzes_olvaso_nev CHECK (LENGTH(olvaso_neve) >= 10)
);
```

3. Feladat: Feltöltés egyszerre (INSERT ALL), idő nélküli dátumokkal (1 pont)

```
INSERT ALL
    -- Legalább 3 könyv:
    INTO konyv (id, cim, kategoria, ar) VALUES (1, 'A Gyűrű Szövetsége', 'Fantasy', 4990)
    INTO konyv (id, cim, kategoria, ar) VALUES (2, 'Alapítvány és Birodalom', 'Sci-fi', 42
        00)
    INTO konyv (id, cim, kategoria, ar) VALUES (3, 'Harry Potter és a Bölcsek Köve', 'Fant
        asy', 5500)
    INTO konyv (id, cim, kategoria, ar) VALUES (4, 'Tiszta Kód', 'Informatika', 8900)
    INTO konyv (id, cim, kategoria, ar) VALUES (5, 'A Da Vinci-kód', 'Krimi', 3900)
    -- Legalább 3 kölcsönzés (TRUNC: tiszta dátum óra/perc/mp nélkül, min. 10 betűs olvasó
        nevek!):
    INTO kolcsonzes (id, konyv_id, olvaso_neve, kivetel_datuma, lejarat_datuma)
        VALUES (101, 1, 'Kovács Jánoska', TRUNC(SYSDATE), TRUNC(SYSDATE) + 14)
    INTO kolcsonzes (id, konyv_id, olvaso_neve, kivetel_datuma, lejarat_datuma)
        VALUES (102, 1, 'Tóth László Béla', TRUNC(SYSDATE) - 5, TRUNC(SYSDATE) + 9)
    INTO kolcsonzes (id, konyv_id, olvaso_neve, kivetel_datuma, lejarat_datuma)
        VALUES (103, 2, 'Szabó-Eszterházy Nóra', TRUNC(SYSDATE), ADD_MONTHS(TRUNC(SYSDAT
        E), 1))
    INTO kolcsonzes (id, konyv_id, olvaso_neve, kivetel_datuma, lejarat_datuma)
        VALUES (104, 3, 'Varga Péter Pál', TRUNC(SYSDATE) - 20, TRUNC(SYSDATE) - 6)
SELECT * FROM dual;

COMMIT;
```

4. Feladat: 20%-os áremelés a már kikölcsönzött könyvekre (2 pont)

```sql
UPDATE konyv
SET ar = ar * 1.2
WHERE id IN (
    SELECT konyv_id
    FROM kolcsonzes
    WHERE konyv_id IS NOT NULL
);

COMMIT;
```

5. Feladat: Könyvek listája, kategória árösszeg + rangsor ablakfüggvénnyel (2 pont)

> 💡 **Megjegyzés:** Mivel minden egyedi könyvnek meg kell jelennie ÉS mellé kell számolni a csoportadatokat, TILOS a sima `GROUP BY` ! Ehelyett az analitikus **ablakfüggvények (`OVER PARTITION BY`)** kellenek!

```sql
SELECT
    cim,
    kategoria,
    ar,
    -- Adott kategóriába tartozó könyvek árainak összege:
    SUM(ar) OVER (PARTITION BY kategoria) AS kategoria_ar_osszeg,
    -- Kategóriánkénti sorszám ár szerint csökkenő sorrendben:
    ROW_NUMBER() OVER (PARTITION BY kategoria ORDER BY ar DESC) AS ar_rangsor_kategorian_b
        elul
FROM konyv
ORDER BY kategoria, ar_rangsor_kategorian_belul;
```

6. Feladat: Kölcsönzési darabszám, 0 kölcsönzés is, ‘a’ betű a címben, darabszám < 3 (3 pont)

> 💡 **Megjegyzések:** - Mivel a sosem kikölcsönzött könyvek is kellenek: `LEFT JOIN` kell! - A számláláshoz `COUNT(kl.id)` kell, NEM `COUNT(*)` (mert az a NULL sorra is 1-et adna)! - Kis- és nagybetű független szűrés: `UPPER(k.cim) LIKE '%A%'` - Csoportszűrés darabszámra: `HAVING COUNT(kl.id) < 3`

```sql
SELECT
    k.cim,
    k.kategoria,
    COUNT(kl.id) AS kolcsonzesek_szama
FROM konyv k
LEFT JOIN kolcsonzes kl ON k.id = kl.konyv_id
WHERE UPPER(k.cim) LIKE '%A%'
GROUP BY k.id, k.cim, k.kategoria
HAVING COUNT(kl.id) < 3
ORDER BY kolcsonzesek_szama DESC, k.cim ASC;
```

Táblamódosítások (ALTER TABLE) - Ha elcseszted a táblát és nem akarod elölről kezdeni

Mert a `CREATE TABLE` után az ember rájön, hogy lehagyott ezt-azt, vagy túl szűk lett egy mező. Nem kell rögtön pánikba esni meg mindent letörölni!

Új oszlop hozzáadása

```
-- Egy új oszlop hozzáadása:
ALTER TABLE tábla_név ADD (új_oszlop VARCHAR2(100));

-- Egyszerre több új oszlop, akár default értékkel:
ALTER TABLE tábla_név ADD (
    ár NUMBER DEFAULT 0,
    leírás VARCHAR2(500)
);
```

Oszlop módosítása (típus, méret, kötelezőség)

```
-- Méret növelése (csökkenteni csak akkor enged az Oracle, ha a meglévő adatok beleférnek):
ALTER TABLE tábla_név MODIFY (megszólítás VARCHAR2(250));

-- Kötelezővé tétel (NOT NULL) - csak akkor sikerül, ha most sincs benne egyetlen NULL értékek sem!:
ALTER TABLE tábla_név MODIFY (megszólítás NOT NULL);

-- Kötelezőség eltörlése (újra lehet benne NULL):
ALTER TABLE tábla_név MODIFY (megszólítás NULL);

-- Alapértelmezett érték megváltoztatása (a meglévő sorokat nem írja át, csak az újakra vonatkozik):
ALTER TABLE tábla_név MODIFY (dátum DEFAULT SYSDATE);
```

Oszlop átnevezése és eldobása

```
-- Oszlop átnevezése:
ALTER TABLE tábla_név RENAME COLUMN megszólítás TO teljes_nev;

-- Oszlop kidobása a kukába:
ALTER TABLE tábla_név DROP COLUMN új_oszlop;
```

Kényszerek (Constraints) utólagos buherálása

```
-- Új CHECK kényszer hozzáadása utólag:
ALTER TABLE tábla_név ADD CONSTRAINT ck_dátum CHECK (dátum >= DATE '1990-01-01');

-- Idegen kulcs hozzáadása kaszkádolt törléssel (ha a szülőt törlöd, a kölke is automatiku
        san repül):
ALTER TABLE másik_tábla_név
ADD CONSTRAINT fk_masik_tabla_szulo
FOREIGN KEY (kŐlső_kulcs_id) REFERENCES tábla_név(azonosító)
ON DELETE CASCADE;

-- VAGY ON DELETE SET NULL (a gyerek megmarad, de a kulcsa NULL-ra vált):
-- FOREIGN KEY (kŐlső_kulcs_id) REFERENCES tábla_név(azonosító) ON DELETE SET NULL;

-- Kényszer ideiglenes KIKAPCSOLÁSA (életmentő, ha ömlesztve akarsz adatot betölteni):
ALTER TABLE tábla_név DISABLE CONSTRAINT ck_dátum;

-- Kényszer visszakapcsolása:
ALTER TABLE tábla_név ENABLE CONSTRAINT ck_dátum;

-- Kényszer végleges törlése:
ALTER TABLE tábla_név DROP CONSTRAINT ck_dátum;
```

Tábla átnevezése, ürítése és eldobása

```
-- Tábla átnevezése:
RENAME régi_tábla_név TO orokolt_tábla_név;

-- TRUNCATE: azonnal kipucolja az összes sort, visszavonhatatlan (nincs ROLLBACK!), de sok
        kal gyorsabb mint a DELETE:
TRUNCATE TABLE regi_tábla_név;

-- DROP TABLE: a tábla megsemmisítése.
-- Ha idegen kulcs hivatkozik rá, CASCADE CONSTRAINTS kell, a PURGE meg kikerüli a lomtára
        t (Recycle Bin):
DROP TABLE regi_tábla_név CASCADE CONSTRAINTS PURGE;
```

Órai mesterfogás: Szótártábla kiépítése meglévő katyvaszból utólag

Az órai `01_dog_manager.sql`-ben van egy zseniális forgatókönyv: van egy táblád (`vacc`), amiben sima szövegként tárolták az oltás kódját (`type_of_vacc`). Ezt akarjuk szépen normalizálni egy külön `vacc_type` szótártáblába, és idegen kulccsal összekötni!

```sql
-- 1. Oszlop átnevezése, hogy jelezze: mostantól ID lesz:
ALTER TABLE vacc RENAME COLUMN type_of_vacc TO vacc_type_id;

-- 2. Új szótártábla létrehozása az elsődleges kulccsal:
CREATE TABLE vacc_type (
    vacc_type_id VARCHAR2(40) NOT NULL,
    vacc_name    VARCHAR2(200),
    CONSTRAINT vacc_type_pk PRIMARY KEY (vacc_type_id)
);

-- 3. Meglévő egyedi kódok átemelése a régi táblából:
INSERT INTO vacc_type (vacc_type_id)
SELECT DISTINCT vacc_type_id FROM vacc;

-- 4. Megnevezések kitöltése:
UPDATE vacc_type SET vacc_name = 'Veszettség elleni oltás' WHERE vacc_type_id = 'A01';
UPDATE vacc_type SET vacc_name = 'Lyme-kór elleni oltás'   WHERE vacc_type_id = 'B01';
UPDATE vacc_type SET vacc_name = 'Kombinált oltás'         WHERE vacc_type_id = 'C01';
COMMIT;

-- 5. Idegen kulcs kényszer ráakasztása a régi táblára:
ALTER TABLE vacc
  ADD CONSTRAINT vacc_type_fk
  FOREIGN KEY (vacc_type_id) REFERENCES vacc_type(vacc_type_id);
```

Bumm! Ezzel a katyvaszból szabályos relációs adatmodell lett adatvesztés nélkül!

DML alapok: UPDATE és DELETE (Mert hibázni emberi dolog)

> **ARANYSZABÁLY:** Mindig legyen `WHERE` feltétel, különben az összes létező sorodat átírod vagy letörlöd a fenébe! És utána `COMMIT;`!

UPDATE (Módosítás)

```sql
-- Egy vagy több mező frissítése adott sornál:
UPDATE tábla_név
SET megszólítás = 'Nagyon Lökött Pista', dátum = SYSDATE
WHERE azonosító = 1;

-- Feltételes módosítás másik tábla adatai alapján (korrelált subquery):
UPDATE másik_tábla_név
SET nem_nulla_szám = nem_nulla_szám * 1.1 -- 10%-os emelés
WHERE kŐlső_kulcs_id IN (SELECT azonosító FROM tábla_név WHERE dátum < DATE '2000-01-01');

COMMIT;
```

DELETE (Törlés)

```
-- Csak a kiválasztott sorok törlése:
DELETE FROM másik_tábla_név
WHERE azonosító = 67;

-- Törlés alkérdezés alapján (pl. töröljük azokat a játékokat, amiknek a gazdája 2000 előt
        t született):
DELETE FROM másik_tábla_név
WHERE kŐlső_kulcs_id IN (SELECT azonosító FROM tábla_név WHERE dátum < DATE '2000-01-01');

COMMIT;
```

Lekérdezések Mesterfokon (SELECT Nagyágyú)

A ZH-k java része ebből áll. Ha ezeket a mintákat felismered, nyert ügyed van.

A SELECT végrehajtási sorrendje (MERT FONTOS!)

Az adatbázis motor **NEM** olyan sorrendben hajtja végre a SELECT-et, ahogy leírod: 1. `FROM` és `JOIN` (összekapcsolja a forrásokat) 2. `WHERE` (soronként szűr még csoportosítás előtt) 3. `GROUP BY` (csoportokba rendezi a sorokat) 4. `HAVING` (csoportokra szűr az aggregált értékek alapján) 5. `SELECT` (kiszámítja a kifejezéseket, oszlopokat és aliasokat) 6. `DISTINCT` (kiszűri a duplikált sorokat) 7. `ORDER BY` (sorba rendezi az eredményt) 8. `FETCH FIRST / ROWNUM` (elvágja a sorok darabszámát)

> **Miért lényeges ez?** Mert a SELECT-ben megadott oszlopálnevet (aliast) NEM használhatod sem a `WHERE`-ben, sem a `GROUP BY`-ban! A WHERE ugyanis hamarabb lefut, mint hogy az álnév egyáltalán megszületne.

Szűrési trükkök a WHERE záradékban

```
-- Szöveges keresés kis/nagybetű függetlenül (UPPER vagy LOWER használatával):
SELECT * FROM tábla_név WHERE UPPER(megszólítás) LIKE '%PISTA%';

-- LIKE helyettesítő karakterek:
-- %  = tetszőleges számú (0 vagy több) karakter
-- _  = pontosan 1 darab tetszőleges karakter
SELECT * FROM tábla_név WHERE megszólítás LIKE '_éni%'; -- Néni, Béni, Péni...

-- Ha konkrétan az aláhúzást vagy a % jelet akarod keresni, ESCAPE kell:
SELECT * FROM tábla_név WHERE megszólítás LIKE '%\_%' ESCAPE '\';

-- Halmazvizsgálat (IN) és intervallum (BETWEEN):
SELECT * FROM tábla_név WHERE azonosító IN (1, 2, 3);
SELECT * FROM tábla_név WHERE dátum BETWEEN TO_DATE('1990/01/01', 'yyyy/mm/dd') AND SYSDAT
        E;

-- NULL ellenőrzés (CSAPDA: soha ne írj olyat, hogy oszlop = NULL! Mindig IS NULL vagy IS
       NOT NULL):
SELECT * FROM tábla_név WHERE megszólítás IS NOT NULL;

-- ☠️ HALÁLOS CSAPDA: NOT IN és a NULL! ☠️
-- Ha az alkérdezés akár egyetlen NULL-t is tartalmaz, a NOT IN SOHA SEMMIT nem ad vissza!
-- EZÉRT MINDIG tegyél bele WHERE oszlop IS NOT NULL szűrést:
SELECT * FROM tábla_név
WHERE azonosító NOT IN (
    SELECT kŐlső_kulcs_id
    FROM másik_tábla_név
    WHERE kŐlső_kulcs_id IS NOT NULL
);
```

JOIN Kisokos (Ki kivel van összekötve?)

Tegyük fel: `tábla_név` = gyerekek, `másik_tábla_név` = játékok.

```sql
-- 1. INNER JOIN (Belső illesztés):
-- Csak azok a gyerekek, akiknek VAN játékuk, és csak azok a játékok, amiknek VAN gazdáju
      k:
SELECT gy.azonosító, gy.megszólítás, j.megszólítás AS jatek_nev
FROM tábla_név gy
INNER JOIN másik_tábla_név j ON gy.azonosító = j.kŐlső_kulcs_id;

-- 2. LEFT (OUTER) JOIN (Bal oldali külső illesztés):
-- MINDEN gyerek szerepel a listában! Akinek nincs játéka, annál a játék oszlopai NULL-ok
      lesznek:
SELECT gy.azonosító, gy.megszólítás, j.megszólítás AS jatek_nev
FROM tábla_név gy
LEFT JOIN másik_tábla_név j ON gy.azonosító = j.kŐlső_kulcs_id;

-- ⭐ ZH KEDVENC: Kik azok a gyerekek, akiknek EGYÁLTALÁN NINCS játékuk?
SELECT gy.azonosító, gy.megszólítás
FROM tábla_név gy
LEFT JOIN másik_tábla_név j ON gy.azonosító = j.kŐlső_kulcs_id
WHERE j.azonosító IS NULL;

-- 3. RIGHT (OUTER) JOIN:
-- MINDEN játék szerepel, még ha nincs is hozzárendelve gyerekhez:
SELECT gy.megszólítás AS gyerek_nev, j.megszólítás AS jatek_nev
FROM tábla_név gy
RIGHT JOIN másik_tábla_név j ON gy.azonosító = j.kŐlső_kulcs_id;

-- 4. FULL (OUTER) JOIN:
-- MINDEN gyerek és MINDEN játék szerepel, hiányzó párok helyén NULL-lal:
SELECT gy.megszólítás AS gyerek_nev, j.megszólítás AS jatek_nev
FROM tábla_név gy
FULL OUTER JOIN másik_tábla_név j ON gy.azonosító = j.kŐlső_kulcs_id;

-- 5. CROSS JOIN (Descartes-szorzat):
-- Minden gyerek minden létező játékkal össze van párosítva (méret = N * M):
SELECT gy.megszólítás, j.megszólítás
FROM tábla_név gy
CROSS JOIN másik_tábla_név j;

-- Régi Oracle stílusú (+) jelölés (ha régebbi feladatsorban látod):
-- Ez pontosan a LEFT JOIN megfelelője (a hiányos oldal mezője mellé kerül a plusz):
SELECT gy.megszólítás, j.megszólítás
FROM tábla_név gy, másik_tábla_név j
WHERE gy.azonosító = j.kŐlső_kulcs_id(+);
```

Csoportosítás (GROUP BY & HAVING)

> **Szabály:** Bármilyen oszlop szerepel a `SELECT`-ben, amit **nem raktál bele aggregáló függvénybe** (mint a `SUM`, `AVG`, `COUNT`, `MIN`, `MAX`), annak **KÖTELEZŐ** szerepelnie a `GROUP BY`-ban!

```sql
SELECT
    kŐlső_kulcs_id                          AS gyerek_id,
    COUNT(*)                                AS jatekok_szama,
    SUM(nem_nulla_szám)                     AS pontszamok_osszege,
    ROUND(AVG(nem_nulla_szám), 2)           AS pontszamok_atlaga,
    MIN(nem_nulla_szám)                     AS legkisebb_pont,
    MAX(nem_nulla_szám)                     AS legnagyobb_pont
FROM másik_tábla_név
WHERE nem_nulla_szám > 0                -- WHERE: sorokat szűr még a csoportosítás ELŐTT
GROUP BY kŐlső_kulcs_id
HAVING COUNT(*) >= 2;                   -- HAVING: csoportokat szűr aggregált érték alapján
       UTÁNA
```

Halmazműveletek (UNION, INTERSECT, MINUS)

Szabály: Mindegyik lekérdezésben az oszlopok számának és típusának kompatibilisnek kell lennie!

```sql
-- UNION: Egyesíti az eredményeket és kiszűri a duplikátumokat (lassabb a rendezés miatt):
SELECT megszólítás FROM tábla_név
UNION
SELECT megszólítás FROM másik_tábla_név;

-- UNION ALL: Egyesíti a sorokat duplikátum-szűrés NÉLKÜL (sokkal gyorsabb!):
SELECT megszólítás FROM tábla_név
UNION ALL
SELECT megszólítás FROM másik_tábla_név;

-- INTERSECT: Metszet (csak azok, amik mindkét halmazban szerepelnek):
SELECT megszólítás FROM tábla_név
INTERSECT
SELECT megszólítás FROM másik_tábla_név;

-- MINUS: Halmazkülönbség (az elsőből kivonja a másodikat):
-- ZH KEDVENC: Mely gyerekeknek NINCS játékuk (MINUS-szal megoldva):
SELECT azonosító FROM tábla_név
MINUS
SELECT kŐlső_kulcs_id FROM másik_tábla_név WHERE kŐlső_kulcs_id IS NOT NULL;
```

Top-N sorok kiválasztása (A legelső N darab rekord)

```
-- 1. Modern módszer (Oracle 12c+):
SELECT *
FROM tábla_név
ORDER BY dátum DESC
FETCH FIRST 3 ROWS ONLY;

-- Lapozás (pl. 3 sor kihagyása után a következő 3 sor):
SELECT *
FROM tábla_név
ORDER BY dátum DESC
OFFSET 3 ROWS FETCH NEXT 3 ROWS ONLY;

-- 2. Régi klasszikus módszer ROWNUM-mal:
-- VIGYÁZAT: Mindig belső almappában (subquery-ben) kell rendezni!
-- Ha simán kiadnád: WHERE ROWNUM <= 3 ORDER BY ..., akkor előbb levesz 3 random sort, és
        csak azt rendezi!
SELECT *
FROM (
    SELECT *
    FROM tábla_név
    ORDER BY dátum DESC
)
WHERE ROWNUM <= 3;
```

CTE (Common Table Expression - A WITH záradék)

Mert az egymásba ágyazott al-lekérdezések átláthatatlan spagettivé válnak:

```
WITH atlag_pontok AS (
    SELECT AVG(nem_nulla_szám) AS atlag
    FROM másik_tábla_név
),
kiemelt_jatekok AS (
    SELECT *
    FROM másik_tábla_név
    WHERE nem_nulla_szám > (SELECT atlag FROM atlag_pontok)
)
SELECT *
FROM kiemelt_jatekok;
```

Neat Tricks & Életmentő Oracle Függvények (ZH Túlélőcsomag)

1. NULL kezelő mesterhármas

```
-- NVL(kifejezés, helyettesítő): ha NULL, a másodikat adja vissza
SELECT NVL(leiras, 'Nincs megadva leírás') FROM kategoria;

-- NVL2(kifejezés, ha_nem_null, ha_null): ternáris operátor
SELECT NVL2(leiras, 'Kitöltött', 'Hiányzik') FROM kategoria;

-- COALESCE(e1, e2, e3, ...): az első nem-NULL értéket adja vissza a felsorolásból
SELECT COALESCE(becenév, megszólítás, 'Névtelen') FROM tábla_név;

-- NULLIF(a, b): ha a = b, akkor NULL, különben az 'a' értéke
SELECT NULLIF(nem_nulla_szám, 0) FROM másik_tábla_név;
```

2. Feltételes logika a SELECT-ben (CASE és DECODE)

```
-- CASE WHEN (bárhol használható, szabványos és olvasható):
SELECT megszólítás,
    CASE
        WHEN dátum < DATE '2000-01-01' THEN 'Régi motoros'
        WHEN dátum < DATE '2015-01-01' THEN 'Fiatal'
        ELSE 'Mai csirke'
    END AS korcsoport
FROM tábla_név;

-- DECODE (Oracle kompakt beépített switch-case):
-- Szintaxis: DECODE(érték, mikor_1, akkor_1, mikor_2, akkor_2, ..., egyébként)
SELECT azonosító,
    DECODE(kŐlső_kulcs_id,
        1, 'Pista tulajdona',
        2, 'Gábor tulajdona',
        'Ismeretlen gazda') AS tulajdonos
FROM másik_tábla_név;
```

3. Dátumvarázslás és Időműveletek

```
-- Aktuális időpont:
SELECT SYSDATE FROM dual;                    -- dátum és idő másodperc pontossággal
SELECT CURRENT_TIMESTAMP FROM dual;          -- mikromásodperc és időzóna

-- Dátum aritmetika napokban (szám hozzáadása = napok száma):
SELECT SYSDATE + 1 FROM dual;                -- holnap ilyenkor (+1 nap)
SELECT SYSDATE - 7 FROM dual;                -- egy héttel ezelőtt
SELECT SYSDATE + (1/24) FROM dual;           -- 1 óra múlva
SELECT SYSDATE + (15/1440) FROM dual;        -- 15 perc múlva (mert egy napban 1440 perc v
        an)

-- Hónapok kezelése (a tanárok kedvence a ZH-n!):
SELECT ADD_MONTHS(SYSDATE, 3) FROM dual;     -- 3 hónap múlva
SELECT ADD_MONTHS(SYSDATE, -6) FROM dual;    -- félévvel ezelőtt
SELECT MONTHS_BETWEEN(SYSDATE, DATE '2024-01-01') FROM dual; -- hány hónap telt el

-- Speciális dátumok:
SELECT LAST_DAY(SYSDATE) FROM dual;          -- az aktuális hónap utolsó napja
SELECT NEXT_DAY(SYSDATE, 'HÉTFŐ') FROM dual; -- következő hétfő

-- TRUNC dátumra (levágja az időt éjfélre vagy adott egységre csonkít):
SELECT TRUNC(SYSDATE) FROM dual;             -- ma 00:00:00 (idő rész nullázva!)
SELECT TRUNC(SYSDATE, 'MM') FROM dual;       -- az aktuális hónap 1. napja
SELECT TRUNC(SYSDATE, 'YYYY') FROM dual;     -- az aktuális év 1. napja

-- Rész kinyerése:
SELECT EXTRACT(YEAR FROM SYSDATE) AS idei_ev FROM dual;
SELECT EXTRACT(MONTH FROM SYSDATE) AS idei_ho FROM dual;

-- Dátum formázás és beolvasás:
SELECT TO_CHAR(SYSDATE, 'YYYY.MM.DD HH24:MI:SS') FROM dual;
SELECT TO_DATE('2026-10-13 14:30', 'YYYY-MM-DD HH24:MI') FROM dual;
```

4. Szövegkezelő függvények (String manipuláció)

```
SELECT UPPER('kisbetű') FROM dual;           -- KISBETŰ
SELECT LOWER('NAGYBETŰ') FROM dual;          -- nagybetű
SELECT INITCAP('kovács jános') FROM dual;    -- Kovács János

-- Részsztring kinyerése (FIGYELEM: 1-től indexel, NEM 0-tól!):
SELECT SUBSTR('Oracle SQL', 1, 6) FROM dual; -- 'Oracle' (1. karaktertől 6 db hosszú)
SELECT SUBSTR('Oracle SQL', -3) FROM dual;   -- 'SQL' (hátulról a 3. karaktertől végig)

-- Hossz és keresés:
SELECT LENGTH('Almafa') FROM dual;           -- 6
SELECT INSTR('Kecskesajt', 'sajt') FROM dual;-- 7 (hányadik karakternél kezdődik a 'sajt')

-- Kitöltés és levágás (kiváló kódok vagy számlák formázásához):
SELECT LPAD('42', 6, '0') FROM dual;         -- '000042' (balról kitöltve 6 hosszúra)
SELECT RPAD('Tétel', 10, '.') FROM dual;     -- 'Tétel.....'
SELECT TRIM('   felesleges szóköz   ') FROM dual;

-- Csere és összefűzés:
SELECT REPLACE('szép nap', 'nap', 'este') FROM dual; -- 'szép este'
SELECT 'Kovács' || ' ' || 'Péter' FROM dual;         -- összefűzés || operátorral
```

5. Számkezelő függvények

```
SELECT ROUND(123.456, 2) FROM dual; -- 123.46 (kerekítés)
SELECT TRUNC(123.456, 2) FROM dual; -- 123.45 (levágás csonkítással, nem kerekít!)
SELECT MOD(10, 3) FROM dual;        -- 1 (maradékos osztás)
SELECT CEIL(4.1) FROM dual;         -- 5 (felső egészrész)
SELECT FLOOR(4.9) FROM dual;        -- 4 (alsó egészrész)
SELECT ABS(-42) FROM dual;          -- 42 (abszolút érték)
SELECT SIGN(-15) FROM dual;         -- -1 (előjel: negatív = -1, nulla = 0, pozitív = 1)
```

6. Analitikus (Ablak-) Függvények (Amikor a GROUP BY már kevés)

Ha úgy akarsz csoportosítani és rangsorolni, hogy közben **nem veszted el az egyedi sorokat**:

```sql
-- Sorszámozás és rangsorolás csoportonként:
SELECT
    kategoria_id, nev, ar,
    -- Simán megszámozza a sorokat 1-től N-ig csoporton belül:
    ROW_NUMBER() OVER (PARTITION BY kategoria_id ORDER BY ar DESC) AS sorszam,
    -- Rangsor holtversennyel és ugrással (1, 2, 2, 4):
    RANK() OVER (PARTITION BY kategoria_id ORDER BY ar DESC) AS rang,
    -- Rangsor holtversennyel de ugrás NÉLKÜL (1, 2, 2, 3):
    DENSE_RANK() OVER (PARTITION BY kategoria_id ORDER BY ar DESC) AS suru_rang
FROM termek;

-- ⭐ ZH KLASSZIKUS: Minden kategóriából a LEGDRÁGÁBB termék kiválasztása:
WITH rangsorolt AS (
    SELECT
        kategoria_id, nev, ar,
        ROW_NUMBER() OVER (PARTITION BY kategoria_id ORDER BY ar DESC) AS rn
    FROM termek
)
SELECT kategoria_id, nev, ar
FROM rangsorolt
WHERE rn = 1;
```

Szekvenciák (Sequences) - Automatikus azonosító számlálók

Mivel régebbi Oracle-ben nem létezett `AUTO_INCREMENT`, szekvenciát kellett létrehozni (ZH-n még ma is kérik!):

```sql
-- Szekvencia készítése:
CREATE SEQUENCE seq_tábla_azonosító
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

-- Használata új sor beszúrásakor (.NEXTVAL lekéri a következő számot):
INSERT INTO tábla_név (azonosító, megszólítás, dátum)
VALUES (seq_tábla_azonosító.NEXTVAL, 'Új Jövevény', SYSDATE);

-- Az aktuális (legutóbb kiadott) érték lekérdezése a munkamenetben:
SELECT seq_tábla_azonosító.CURRVAL FROM dual;

-- Szekvencia törlése:
DROP SEQUENCE seq_tábla_azonosító;
```

(Oracle 12c-től már létezik közvetlen Identity oszlop is: `azonosító NUMBER GENERATED ALWAYS AS IDENTITY`)

Indexek (Ha száguldani kell a WHERE-ben)

```
-- Sima B-fa index létrehozása oszlopra:
CREATE INDEX idx_termek_nev ON termek(nev);

-- Egyedi index (nem enged azonos értékeket):
CREATE UNIQUE INDEX uq_idx_termek_nev ON termek(nev);

-- Összetett index több oszlopra:
CREATE INDEX idx_termek_kat_ar ON termek(kategoria_id, ar);

-- Index kidobása:
DROP INDEX idx_termek_nev;
```

Tranzakciókezelés: COMMIT, ROLLBACK és Mentési Pontok

```
-- Módosítunk valamit:
UPDATE tábla_név SET megszólítás = 'Próba Név' WHERE azonosító = 1;

-- Mentési pont létrehozása (időbélyeg a tranzakcióban):
SAVEPOINT mentes_1;

-- Csinálunk valami hülyeséget véletlenül:
DELETE FROM tábla_név WHERE azonosító = 1;

-- Visszagörgetünk a mentési pontig (a DELETE visszavonódik, de a korábbi UPDATE megmara
      d!):
ROLLBACK TO mentes_1;

-- Véglegesítjük a tranzakciót (innentől a változások beégnek az adatbázisba):
COMMIT;

-- Ha mindent vissza akarsz vonni a legutolsó COMMIT óta:
-- ROLLBACK;
```

> **Figyelem:** Minden DDL utasítás (`CREATE` , `ALTER` , `DROP` , `TRUNCATE`) előtt és után az Oracle **AUTOMATIKUSAN COMMIT-ot hajt végre**! Ezért egy `CREATE TABLE` után már nem tudsz `ROLLBACK` -et nyomni a korábbi adataidra!

Adatszótár Puskák (Data Dictionary Cheat Sheet)

Amikor a ZH közben nem emlékszel a táblád, oszlopod vagy kényszered pontos nevére, ezeket futtasd le:

```
-- 1. Milyen tábláim vannak a sémámban?
SELECT table_name, num_rows FROM user_tables ORDER BY table_name;

-- 2. Milyen oszlopai és adattípusai vannak a táblámnak?
SELECT column_name, data_type, data_length, nullable, data_default
FROM user_tab_columns
WHERE table_name = 'TERMEK'   -- FONTOS: csupa nagybetűvel írd a nevet!
ORDER BY column_id;

-- 3. Milyen kényszerek vannak beállítva a táblán?
SELECT constraint_name, constraint_type, search_condition, status
FROM user_constraints
WHERE table_name = 'TERMEK';
-- Kényszertípusok:
--   P = Primary Key
--   R = Referential (Foreign Key)
--   U = Unique
--   C = Check (vagy Not Null)

-- 4. Melyik oszlophoz tartozik a kényszer?
SELECT constraint_name, column_name, position
FROM user_cons_columns
WHERE table_name = 'TERMEK';

-- 5. Milyen nézeteim (VIEW) vannak és mi a forráskódjuk?
SELECT view_name, text FROM user_views;

-- 6. Milyen szekvenciáim vannak és hol áll a számláló?
SELECT sequence_name, last_number, increment_by FROM user_sequences;

-- 7. Milyen indexeim vannak?
SELECT index_name, table_name, uniqueness FROM user_indexes;
```

A Leggyakoribb Oracle Hibák és Megoldásuk (Pánikgomb)

Ha pirossal dobja a hibát az SQL Developer, nézd meg itt:

Hibakód	Hibaüzenet	Mi történt és mi a megoldás?
ORA-65096	`invalid common user name`	A gyökér konténerhez (CDB) kapcsolódtál a PDB helyett! Futtasd: `ALTER SESSION SET CONTAINER = XEPDB1;`
ORA-00942	`table or view does not exist`	Elgépelted a tábla nevét, vagy nem a megfelelő felhasználói kapcsolathoz tartozó munkalapon vagy.
ORA-00001	`unique constraint (...) violated`	Olyan értéket akarsz beszúrni, ami az elsődleges kulcsban (`PK`) vagy egyedi (`UNIQUE`) mezőben már létezik.
ORA-02291	`integrity constraint (...) violated - parent key not found`	Külső kulcs (`FK`) hiba: olyan szülő ID-re hivatkozol beszúráskor, ami nem létezik a hivatkozott táblában!
ORA-02292	`integrity constraint (...) violated - child record found`	Törölni akarsz a szülő táblából egy sort, amire a gyerek táblában még mutat külső kulcs! Töröld előbb a gyereket vagy használj `ON DELETE CASCADE` -et.
ORA-01400	`cannot insert NULL into (...)`	Nem adtál meg értéket egy olyan mezőnél, amire `NOT NULL` kényszer van érvényben.
ORA-01722	`invalid number`	Szám típusú mezőbe nem konvertálható szöveget próbálsz beletuszkolni.
ORA-00937	`not a single-group group function`	A SELECT-ben van aggregáló függvény (`COUNT` , `SUM` stb.) és sima oszlop is, de a sima oszlopot elfelejtetted felvenni a `GROUP BY` -ba!
ORA-00918	`column ambiguously defined`	Két összekapcsolt táblában azonos nevű oszlop szerepel, és a SELECT-ben nem írtad elé a tábla nevét/aliasát (pl. `id` helyett `k.id` vagy `t.id`).
ORA-00933	`SQL command not properly ended`	Szintaktikai hiba: felesleges vagy hiányzó vessző, rossz helyre tett kulcsszó vagy pontosvessző alkérdezés belsejében.
ORA-00054	`resource busy and acquire with NOWAIT specified`	Egy másik session vagy tranzakció zárolta a sort/táblát (elfelejtett `COMMIT` miatti lock). Menj a másik

Hibakód	Hibaüzenet	Mi történt és mi a megoldás?
		munkalapra és adj ki egy `COMMIT;`-ot!

PL/SQL Túlélőcsomag (Órai Anyag & 2. ZH Előzetes)

Az órai anyagban (`02_add_vacc_type.prc` -től `09_dbms_output.sql` -ig) átléptünk a sima SQL-ből a PL/SQL (Procedural Language / SQL) világába. Ha a tanár a 2. ZH-n vagy órán ezt kéri, ezek a kötelező túlélő alapok:

1. Az Aranyszabály: Kapcsold be a kimenetet!

Ha nem látod a konzolon a `DBMS_OUTPUT.PUT_LINE` kiírásait, az nem azért van mert hülye a kód, hanem mert alapból le van tiltva a kimenet!

```
SET SERVEROUTPUT ON;
```

(SQL Developerben: Nézet (View) -> Dbms Output panel megnyitása és zöld plusz gomb a kapcsolatra!)

2. Névtelen blokk (Anonymous Block) szerkezete

```
DECLARE
    -- Változók, konstansok, kurzorok deklarációja
    v_szam NUMBER := 10;
    v_szoveg VARCHAR2(100) := 'Szevasz világ!';
BEGIN
    -- Végrehajtható kód
    DBMS_OUTPUT.PUT_LINE(v_szoveg || ' Szám értéke: ' || v_szam);
EXCEPTION
    -- Hibakezelés (ha valami elszáll)
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Gáz van: ' || SQLERRM);
END;
/
```

(A végén a `/` jel elengedhetetlen, az indítja el a futtatást!)

3. A Szentséges SELECT ... INTO és a Két Mumus

Amikor egy SQL lekérdezés eredményét PL/SQL változóba akarod tölteni:

```
DECLARE
    v_nev VARCHAR2(100);
BEGIN
    SELECT megszólítás INTO v_nev
    FROM tábla_név
    WHERE azonosító = 1;

    DBMS_OUTPUT.PUT_LINE('Megtalált név: ' || v_nev);
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('Egyetlen árva sor sincs ilyen ID-vel!');
    WHEN TOO_MANY_ROWS THEN
        DBMS_OUTPUT.PUT_LINE('Baj van: több sort adott vissza a lekérdezés!');
END;
/
```

> ☠️ **SZABÁLY:** A `SELECT ... INTO` kizárólag **PONTOSAN 1 sort** adhat vissza! Ha 0 jön vissza: `NO_DATA_FOUND`. Ha 2 vagy több: `TOO_MANY_ROWS`.

4. Tárolt Eljárások (PROCEDURE) - IN és OUT paraméterekkel

Egyszerű beszúró eljárás (órai `02_add_vacc_type.prc`):

```
CREATE OR REPLACE PROCEDURE add_vacc_type(
    p_vacc_type_id IN VARCHAR2,
    p_vacc_name    IN VARCHAR2
) IS
BEGIN
    INSERT INTO vacc_type (vacc_type_id, vacc_name)
    VALUES (p_vacc_type_id, p_vacc_name);
    COMMIT;
END add_vacc_type;
/
```

Több értéket visszaadó (OUT paraméteres) eljárás (órai `08_all_dog_details.prc`):

```
CREATE OR REPLACE PROCEDURE all_dog_details(
    p_min_weight OUT NUMBER,
    p_max_weight OUT NUMBER,
    p_avg_weight OUT NUMBER
) IS
BEGIN
    SELECT MIN(birth_weight), MAX(birth_weight), AVG(birth_weight)
    INTO p_min_weight, p_max_weight, p_avg_weight
    FROM dog;
END all_dog_details;
/
```

Hogyan hívjuk meg és teszteljük az eljárást névtelen blokkból? (órai `09_dbms_output.sql`):

```
DECLARE
    v_min NUMBER;
    v_max NUMBER;
    v_avg NUMBER;
BEGIN
    -- Eljárás hívása nevesített paraméterátadással (=>):
    all_dog_details(
        p_min_weight => v_min,
        p_max_weight => v_max,
        p_avg_weight => v_avg
    );

    DBMS_OUTPUT.PUT_LINE('Min súly: ' || v_min || chr(10) ||
                         'Max súly: ' || v_max || chr(10) ||
                         'Átlag súly: ' || ROUND(v_avg, 2));
END;
/
```

5. Tárolt Függvények (FUNCTION)

A függvény és az eljárás közötti kulcskülönbség: a függvény **MINDIG visszatér egy értékkel** (`RETURN`), és közvetlenül beágyazható akár egy sima `SELECT` -be is!

```
-- Órai `get_prev_date` minta:
CREATE OR REPLACE FUNCTION get_prev_date(p_n NUMBER DEFAULT 1)
RETURN DATE IS
BEGIN
    RETURN SYSDATE - p_n;
END get_prev_date;
/

-- Használata sima lekérdezésben:
SELECT get_prev_date(7) AS egy_hettel_ezelott FROM dual;
```

6. PL/SQL Csomagok (PACKAGE) - A kód modularitása

Egy csomag mindig két részből áll: 1. **Package Specification (Fejléc / Interfész):** Csak a publikusan elérhető függvények, eljárások és változók listája. 2. **Package Body (Törzs):** A tényleges megvalósítás és a privát (rejtett) változók.

```
-- 1. SPECIFIKÁCIÓ (órai 06_pkg_my_package.pck minta):
CREATE OR REPLACE PACKAGE pkg_szamlalo IS
    gv_increment NUMBER;                    -- Globális, kívülről is elérhető változó
    FUNCTION next_number RETURN NUMBER;     -- Publikus függvény
END pkg_szamlalo;
/

-- 2. TÖRZS:
CREATE OR REPLACE PACKAGE BODY pkg_szamlalo IS
    gv_number NUMBER;                       -- Privát belső változó (kívülről rejtve)

    FUNCTION next_number RETURN NUMBER IS
    BEGIN
        gv_number := gv_number + gv_increment;
        RETURN gv_number;
    END next_number;

-- Inicializációs blokk (a session-ben az első csomaghíváskor automatikusan lefut!):
BEGIN
    IF gv_increment IS NULL THEN
        gv_increment := 1;
    END IF;
    gv_number := 0;
END pkg_szamlalo;
/
```

> 💡 **Csomagok szuperereje:** A csomagváltozók megtartják az állapotukat a munkamenet (session) teljes hossza alatt! Ha 3-szor meghívod a `pkg_szamlalo.next_number`-t, az rendre 1, 2, 3-at fog adni!

7. Audit és Történeti Naplózás Eljárásban (A `change_owner` Minta)

Az órai `04_dog_owner_hist.sql` és `05_change_owner.prc` klasszikus minta: mielőtt felülírsz egy értéket, mentsd el a régit egy naplótáblába!

```
-- 1. Történeti tábla:
CREATE TABLE kutya_gazda_naplo (
    kutya_id   NUMBER    NOT NULL,
    regi_gazda NUMBER    NOT NULL,
    mod_ideje  TIMESTAMP NOT NULL
);

-- 2. Gazdaváltó eljárás naplózással:
CREATE OR REPLACE PROCEDURE gazda_modositas(
    p_kutya_id    NUMBER,
    p_uj_gazda_id NUMBER
) IS
    l_regi_gazda NUMBER;
BEGIN
    -- 1. Lépés: Lekérjük a jelenlegi gazdát
    SELECT owner_id INTO l_regi_gazda
    FROM dog
    WHERE dog_id = p_kutya_id;

    -- 2. Lépés: Átírjuk az újra a kutyánál
    UPDATE dog
    SET owner_id = p_uj_gazda_id
    WHERE dog_id = p_kutya_id;

    -- 3. Lépés: Elmentjük a régi állapotot a naplótáblába időbélyeggel
    INSERT INTO kutya_gazda_naplo (kutya_id, regi_gazda, mod_ideje)
    VALUES (p_kutya_id, l_regi_gazda, SYSTIMESTAMP);

    COMMIT;
END gazda_modositas;
/
```

ZH Környezet & Auditáló Túlélési Szabályzat (Hogyan ne ess ki a játékból az első 5 percben)

A vizsga szigorúan felügyelt környezetben zajlik az `auditalo.exe` segítségével. Ha nem figyelsz a formai szabályokra, még a kódírás előtt megbukhatsz:

1. **Semmi ne legyen nyitva:** Mielőtt elindítanád az `auditalo.exe`-t, zárj be MINDEN böngészőt (Chrome, Edge, Firefox)! Semmilyen fül nem futhat a háttérben.
2. **Mappa elnevezése pontosan:** Hozz létre egy mappát a megadott séma szerint: `<vezeteknev_keresztnev_neptunkod>_adatb2_zh1` *(Pót ZH esetén a végén* `_zh1_pot` *szerepeljen!)*
3. **Auditáló indítása:**
   - Másold a mappába az `auditalo.exe`-t és indítsd el.
   - Ha a Windows Defender akadékoskodik: *További információ -> Futtatás mindenképpen*.
   - Felugrik egy fekete konzolablak (Command Window): **EZT TILOS BEZÁRNI A TELJES ZH ALATT!**
4. **A Két Mappa szigorú szabálya:**
   - `zh_audit`: **SZIGORÚAN TABU!** Ne nyisd meg, ne módosítsd, ne töröld, mert fegyelmi jár érte!
   - `zh`: **CSAK ITT DOLGOZZ!** Végig ebbe a mappába mentsd a munkádat!
5. **Fájlok szervezése:**
   - Minden egyes feladathoz csinálj külön `.sql` fájlt (pl. `feladat1.sql`, `feladat2.sql`, ...).
   - Nyisd meg őket az SQL Developerben közvetlenül a `zh` mappából, és írás közben nyomkodj folyamatosan `Ctrl + S`-t!
6. **Böngésző szabály:**
   - Az auditáló elindít egy saját Chrome böngészőt.
   - **KIZÁRÓLAG EZT A BÖNGÉSZŐT SZABAD HASZNÁLNI!** Bármilyen külső böngésző vagy inkognitó ablak megnyitása azonnali diszkvalifikációt jelent!
7. **Beadás:**
   - A legvégén a teljes `<vezeteknev_keresztnev_neptunkod>_adatb2_zh1` mappát tömörítsd be ZIP-be és töltsd fel a Moodle-ba. A ZIP-nek tartalmaznia kell az exe-t, a `zh` és a `zh_audit` mappákat is.